Lab 5 - Topological Sort

Course: CIS 387 - Analysis of Algorithms - Fall 2026
Instructor: Dr. Aaron Mininger

Submission Instructions

Due Date: Nov 17 at 11:59am (beginning of class!)

To Submit: Upload your completed **`PackageDependencies.java`** file, plus the image you generated.

Since this is a lab assignment, you are allowed to work with other people. However, you must type up and submit your own individual work. You may consult other resources, including AI, to write the code, but try to work through things yourself first.

Setup

All the code and files needed for this homework are available on the course git repository:
https://github.com/amininger-mu/CIS387_F26_Code

In the **`labs/lab5_topo_sort`** folder, you will see provided supporting code.

---

1. Project Structure

This section gives an overview of the supporting code in the project. You will not need to edit these files, but should be familiar with them.

`Graph.java` and **`Digraph.java`**

These are 2 interfaces we use for the graph algorithms.

```
public interface Graph
```

`int V()`	*Returns the number of vertices*
`int E()`	*Returns the number of edges*
`void addEdge(int v, int w)`	*Adds an edge from v -> w*
`Iterable<Integer> vertices()`	*Returns an iterable view of the vertices (0 to V-1)*
`Iterable<Integer> adj(int v)`	*Returns an iterable view of the vertices adjacent to v*

```
public interface Digraph extends Graph
```

`Digraph reverse()`	*Returns a copy of the digraph, but with all edges reversed*

`AdjacencyListDigraph.java`

This class implements the **`Digraph`** interface using adjacency lists.

`SymbolDigraph.java`

Sometimes we want to make a graph where the vertices are not integers but correspond to something else. Here we use a **`SymbolDigraph`**, which lets us use Strings as vertex names.

`public class SymbolDigraph`

`SymbolDigraph(File file, String sep)`	*Creates a new symbol digraph by reading the given file*
`String contains(String s)`	*Returns true if the given String is a vertex in the graph*
`int indexOf(String s)`	*Returns the vertex number of the given String*
`String nameOf(int v)`	*Returns the String (name) of the given vertex number*

You initialize the symbol graph by reading a file, where it builds up a mapping from String names to vertex indices and vice-versa. You can get this mapping via:

- **`int indexOf(s)`** returns the vertex id associated with the given String name
- **`String nameOf(v)`** returns the String name associated with vertex **`v`**

It is initialized by reading a file where each line contains a vertex name, followed by all it's outgoing edges (adjacent vertices), where vertices are separated by the **`sep`** String

`DirectedCycle.java`

This class implements the algorithm discussed in class to check if a graph is acyclic.

`public class DirectedCycle`

`DirectedCycle(Digraph G)`	*Runs a cycle check algorithm on the given digraph*
`boolean hasCycle()`	*Returns true if a cycle was found in the given graph*
`Iterable<Integer> cycle()`	*Returns an iterable view of the cycle found during search*

2. Lab 5 Instructions

In this lab, we will study the problem of resolving software dependencies. When installing a new piece of software or a library, it usually depends on other software being present or installed first.

For example, with python and pip, if I want to install the **`requests`** python package, it depends on other python packages:

```
pip show requests
Requires: certifi, charset_normalizer, idna, urllib3
```

Problem: Given a dataset of Ubuntu software package dependencies, and a particular package to install, determine which other packages are required and an install order that respects dependencies.

We can solve this problem via a **topological sort!** By doing a depth first search, we can find all the dependencies for a given package. And by getting the topological order, we can ensure that every package is installed after its dependencies.

Data

I have provided a file called **`ubuntu.txt`**, which shows a list of all the packages in Ubuntu's main package repository, plus their dependencies.

For example, on line 13, we see:

```
alsa-base,kmod,linux-sound-base,udev
```

This tells us that the **`alsa-base`** package (a library for handling sound) depends on packages **`kmod`**, **`linux-sound-base`**, and **`udev`**. (And each of these may have their own dependencies.)

Getting Depth First Search Order

Remember that "*Reverse postorder in a DAG is a topological sort*"

To get this, we have to modify our DFS code. Open **`DepthFirstSearch.java`** and see the standard **`dfs`** implementation. We need to modify it so that it reports the preorder and postorder lists.

I have done most of the work for you: I added two ArrayLists **`pre`** and **`post`**, plus some getters.

Your Step: Modify the **`dfs`** so that it adds the vertex **`v`** to the preorder list at the beginning of the function, and adds **`v`** to the postorder list at the end of the function.

Main Program

Write a program called **`PackageDependencies`** that takes the name of a package, and prints out a safe dependency install order for that package.

Write the program **`PackageDependencies.java`** by following these steps:

1. Create a **`SymbolDigraph`** using the file **`ubuntu.txt`** and comma separator **`","`**
2. Extract the **`Digraph`** from the symbol graph using **`.digraph()`**
3. Get the package name from **`args[0]`** and check that it exists in the given symbol graph.
   - If not, print an error message and quit.
4. Use the provided **`DirectedCycle`** class to make sure there is not a cycle in the digraph.
   - If a cycle exits, print an error message and quit.
5. Perform a **`DepthFirstSearch`** starting at the given package's vertex index.
6. Print out the topological order by getting the **`reversePostorder`** from the DFS.
   - You must print the package names, not indices

For example, here is correct output for **`java PackageDependencies linux-sound-base`**

- Note that you would install the packages in the opposite order

```
linux-sound-base
kmod
libkmod2
libzstd1
libssl3t64
liblzma5
libc6
debconf
```

Visualize

We are going to use a tool called Graphviz to visualize our dependency graph.

At the end of main, call the provided function **`exportDotFile`**. This will write a file called **`packages.dot`** containing the information about the dependency graph. Then, run your program to analyze the package **`nginx`** (a popular HTTP web server).

Go to **`https://dreampuf.github.io/GraphvizOnline/`** and paste in the contents of the **`packages.dot`** file. Then download the result as an image and include it in your submission.